Chapter 6
Identification and Classification of Cyber Threats Through SSH Honeypot Systems

José María Jorquera Valero
Department of Information and Communications Engineering, University of Murcia, Spain

Manuel Gil Pérez
Department of Information and Communications Engineering, University of Murcia, Spain

Alberto Huertas Celdrán
https://orcid.org/0000-0001-7125-1710
Telecommunications Software and Systems Group, Waterford Institute of Technology, Ireland

Gregorio Martínez Pérez
Department of Information and Communications Engineering, University of Murcia, Spain

ABSTRACT

As the number and sophistication of cyber threats increases year after year, security systems such as antivirus, firewalls, or Intrusion Detection Systems based on misuse detection techniques are improved in detection capabilities. However, these traditional systems are usually limited to detect potential threats, since they are inadequate to spot zero-day attacks or mutations in behaviour. Authors propose using honeypot systems as a further security layer able to provide an intelligence holistic level in detecting unknown threats, or well-known attacks with new behaviour patterns. Since brute-force attacks are increasing in recent years, authors opted for an SSH medium-interaction honeypot to acquire a log set from attacker's interactions. The proposed system is able to acquire behaviour patterns of each attacker and link them with future sessions for early detection. Authors also generate a feature set to feed Machine Learning algorithms with the main goal of identifying and classifying attacker's sessions, and thus be able to learn malicious intentions in executing cyber threats.

DOI: 10.4018/978-1-7998-2242-4.ch006

Copyright © 2020, IGI Global. Copying or distributing in print or electronic forms without written permission of IGI Global is prohibited.

1. INTRODUCTION

Traditional security systems based on signatures, such as firewalls, antivirus systems or Intrusion Detection Systems (IDS) based on *misuse detection* techniques (Depren, Topallar, Anarim & Ciliz, 2005), are generally not quite effective against unknown threats or new attack patterns not previously registered. However, IDSs based on *anomaly detection* techniques are able to recognise unknown threats, but they are prone to generate false positives –alerts generated when identifying the possible existence of a real threat, when they are actually false alerts because the threat is not true. To cover this need and try to get information of novel threats or attacks, honeypots (Spitzner, 2003) arise, which consist of lure or trap systems whose main goal is to simulate a real system that can be attacked. Through gathered information, it is possible to adapt the IDSs with new detection signatures (those based on misuse detection) or label datasets used by those based on anomaly detection (Jorquera et al., 2018) to retrain their search engine, and, therefore, to increase their precision and recall by reducing the number of false alerts. In addition, one of the main objectives of honeypots is to complement the different traditional techniques in threats detection with a system capable of detecting previously unknown threats, thereby providing a higher level of security (Olakanmi & Dada, 2019).

Honeypots, as the name suggests, are designed to attract the attention of attackers or hackers, so that their efforts to attack and cause serious damage to real systems will be diminished because they are running in a virtual environment. These systems simulate networks and application services such as Report Desktop Protocol (RDP), Secure Shell (SSH), File Transfer Protocol (FTP) and Telnet, amongst others. From the attacker's standpoint, the honeypot system is perceived as a vulnerable execution server that can be used to enter in servers that are currently operational and execute any kind of malicious action.

One of the primary advantages of honeypot systems over traditional systems is that honeypots are able to detect zero-day attacks (Zhang, Wang, Jajodia, Singhal & Albanese, 2016), that is, attacks on vulnerabilities that have not been patched or made public. Another benefit of using honeypots is that they only gather data in case an attack is being generated. Therefore, these systems are able to register a reduced dataset, which involves a lower cost in the system and a lower resource consumption by producing a lower number of alerts that will avoid, in some cases, that administrators can ignore alerts due to the existence of a higher alerts amount and a higher rate of false alerts. In the case of honeypots, whatever access to the system is not normally authorised, therefore, through its use enterprises and organisations can decrease the number of false alerts registered. Finally, another benefit supplied by these systems is to help Information Technology (IT) teams identify vulnerabilities in active servers.

Since there are many honeypot systems that simulate different network and application services, this chapter is focused on Medium-Interaction Honeypots (MIH) (Fraunholz, Krohmer, Anton & Schotten, 2017), that is, honeypots that allow capturing and registering logs about activities and executions from attackers without compromising production systems, based on the SSH (Barrett, Silverman & Byrnes, 2005) network service, being this one the most used by attackers to compromise systems exposed on the Internet. In this subcategory, there are several honeypots that provide similar functionalities amongst others. Since many devices exposed to the Internet are based on Unix systems, one of the most popular honeypot systems is Cowrie (Oosterhof, 2019), which records attacker's commands execution on the system. Another of the main reasons so that this software has been selected is due to its capability to monitor activities and executions carried out by attackers. Furthermore, Cowrie is a honeypot based on SSH, being the brute-force attack the most repeated one when a system is threatened (eSentire, 2017). One more determinant reason so that Cowrie is selected is because amongst all available honeypots of

the same family, such as Kippo (Valli, Rabadia & Woodward, 2013) or Kojoney2 (Keane, 2019), Cowrie is currently the most up-to-date SSH-based MIH.

In spite of benefits provided by honeypot systems, these also have a set of challenges that should be tackled to supply an effective system for detecting previously unregistered threats, as well as discovering new patterns of known threat attacks. Some of the challenges to address are an appropriate configuration of the trap system, the risk determination under which our real production system is being exposed, according to the interaction level of the honeypot system, and a suitable features selection to identify threats from the logs generated by honeypots.

With the aim of generating a system that meets the most relevant challenges considered above, this chapter consists of researching, designing and implementing a system capable of classifying the attacker's behaviour on the basis of actions taken (for example, execution of commands) during the duration of their sessions in a protected and simulated environment, such as the one provided by the Cowrie SSH honeypot. Since a honeypot is not capable of carrying out the threat detection phases based on the information that will be gathered in its logs, this work extracts the most relevant features of each session, providing a layer of intelligence (Jiang, Fu, Gupta, Lou, Rho, Meng & Tian, 2018) to Cowrie through various Machine Learning (ML) techniques (Gupta & Sheng, 2019). For this purpose, different ML techniques based on supervised and unsupervised learning have been used with the main objective of classifying similar behaviours between different sessions. In addition, the system allows storing, analysing and visualising the data provided by each attacker during his/her interaction, through tools such as the ElasticSearch search engine (Rosenberg, Coronel, Meiring, Gray & Brown, 2019) or the Kibana data viewer (Sharma, 2017). Likewise, the presented work can be used as a future basis for the subsequent generation of detection signatures with which to update systems based on misuse detection, or the labelling of datasets for detection engines retraining of systems based on anomaly detection.

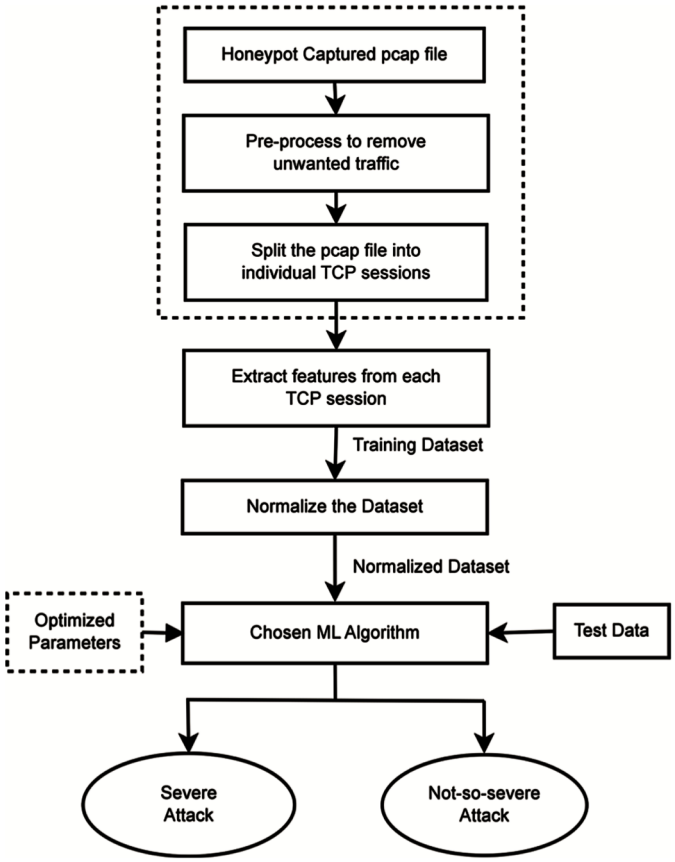
The rest of this chapter is structured as follows. Section 2 reviews previous works in the same field, focusing attention on proposals based on SSH-based MIHs, as well as solutions that present relevant results from logs generated. Section 3 explains the methodology and the main objectives of the proposed solution. Next, the different experiments performed to improve the final solution are explained in Section 4, while in Section 5 we compare the proposed system with others of the literature and then discuss its possible limitations. Finally, Section 6 summarizes the work carried out and proposes different future steps to continue with the development of the system.

2. LITERATURE REVIEW

In this section, we provide a detailed analysis of the current state-of-the-art for SSH-based MIH systems, as well as several experiments carried out by previous researchers that have been published in different scientific articles and are based on the detection of threats using different honeypots.

In Sadasivam, Hota & Anand (2018), the authors carried out an investigation to differentiate the SSH attacks received in Kippo, a honeypot that was taken as a base for the development of Cowrie with new characteristics, allowing to classify the cyber threats (Gupta, 2018) in two categories (see Figure 1). The first category, also known as *severe attacks*, identifies any session type that logs one or more commands into the system, using a compromised SSH server. The second category includes attacks related to SSH port scans, SSH brute-force attacks and brute-force attacks that are successful but are not subsequently performed on the system. The second category is called *not-so-severe attacks*. Some of the features that

Figure 1. Phases for features acquisition and determination of attacks with Kippo

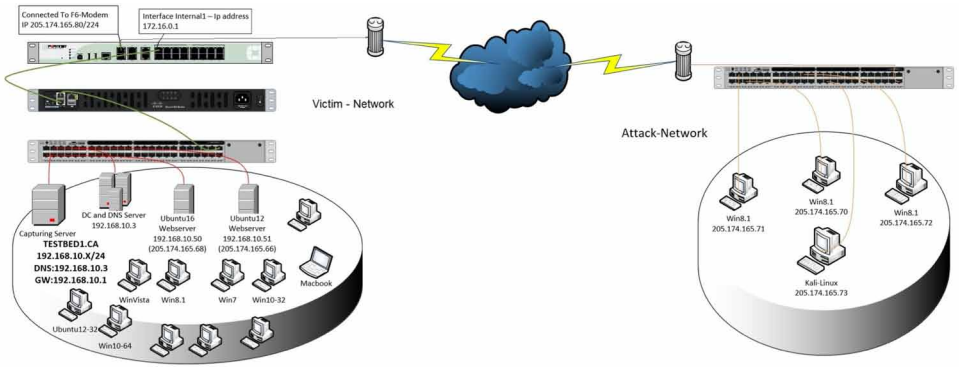


they selected to classify sessions are based on TCP streams, such as the total number of packets received or sent, or the sum of all bytes containing the received payloads. From these features, and using ML algorithms such as k-NN, J48 or PART, the authors obtained in the best case (using J48 and PART) a precision of 1.0, an accuracy of 0.99 and an F2-score of 0.942. Despite obtaining remarkable performance measures to classify SSH attacks according to the nature of the attack, the research provided by authors is not able to determine what type of attack it was.

In a subsequent publication, malicious traffic captured on the network from brute-force attacks on the Kippo honeypot was analysed in (Sadasivam, Hota & Anand, 2018b). Its main purpose was to propose a methodology to segregate individual botnets from network traffic, allowing a more detailed understanding of distributed brute-force attacks that provided a way to detect these attacks. For this purpose, authors analysed the captured TCP flow and determined relevant features that allow linking different traffics to the same botnet. For example, in distributed brute-force attacks, most programs that use a bot are written in the same programming language, SSH client and cryptographic algorithm, and it is possible to use these features to select a set of hots infected by the same botmaster, or even if each of the login credentials are different it is highly likely that this set of IPs forms a botnet.

On the other hand, the research tackled by (Dowling, Schukat & Melvin, 2017) has as its main objective to attract as much malicious traffic as possible and to observe whether the attacker is interested

Figure 2. Network architecture built by the Canadian Institute for Cybersecurity

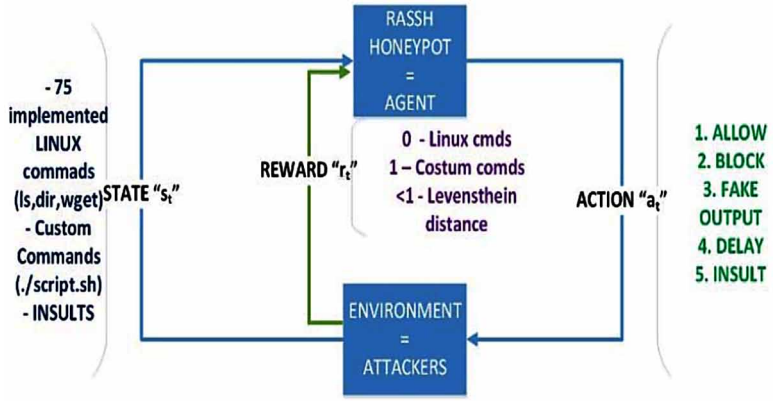


in ZigBee networks. For this, the authors made use of the Kippo SSH honeypot. With this honeypot, attackers can find fictitious ZigBee medical traffic on Wireless Sensor Networks and honeytokens (Padayachee, 2014) pointing to another server, where the researchers had deployed a honeynet. In their testing environment, authors claimed that about 94% of the attacks recorded are dictionary-based attacks. In addition, they claimed that each brute-force detected attack triggered a system reconnaissance attack. It worth mentioning that amongst the attacks that can be detected based on logs we can highlight dictionary-based attacks, brute-force attacks, launched attacks or individual attacks, where the latter shows certain interest in honeytokens and trying to manipulate them, recognition scripts and botnets.

Another study based on attacks on honeypots is the one presented by Sharafaldin, Lashkari & Ghorbani (2018). The authors, who conducted the research in conjunction with the Cybersecurity Institute of Canada, analysed TCP traffic entering their honeypots to obtain a feature set that can detect attacks categories. The test environment consists of a machine set that make up the victim network and a set of 4 computers with different IPs that make up the attackers (see Figure 2). In the case of the attackers, they used both Linux and Windows systems, while in the case of the victims the main operating systems were macOS, Linux and Windows. These authors carried out all the necessary tests in their own controlled environment for 5 days. Each day, specific attacks were performed which were afterwards used to analyse and label the data flows. After testing, they managed to generate a classification for 5 types of attacks: brute-force attacks, Denial of Service (DoS) attacks and Distributed Denial of Service (DDoS) attacks, web attacks, infiltration attacks and botnets. Finally, through the use of classification algorithms such as k-NN and Random Forest, they were able to obtain an accuracy of 0.96 and 0.98, a recall of 0.96 and 0.97, and an F1-score of 0.96 and 0.97, respectively. A limitation of this approach is to face changes in the attack patterns of already known threats such as botnets, which periodically change their activity and their Command & Control (C&C) location mechanisms so that they are not easily detected.

The study carried out by Pauna & Bica (2014) presents a novel implementation of an adaptive honeypot system, which follows the approach used by the Heliza adaptive honeypot, improving several of its disadvantages. The proposed solution, also known as Reinforced Adaptive SSH Honeypot (RASSH), uses the Kippo honeypot code as a base, simulating an SSH server that improves functionalities, in particular providing greater learning capabilities and scalability. This solution improved the Kippo honeypot through the implementation of several new commands, this being one of main reasons why Cowrie was developed later. RASSH is able to interact with attackers through reinforcement learning,

Figure 3. Reinforced learning scheme and available RASSH actions



that is, trying to solve the problem faced by an agent when it must learn the behaviour when interacting with a dynamic environment through a test and error mechanism. In addition, the system proposed by Pauna & Bica (2014) develops an intermediate module between a honeypot and the learning layer by reinforcement that allows offering to attackers a wide set of actions (see Figure 3) that will be activated based on the type of commands that the attacker typed. This idea is another improvement to highlight, since one of the main reasons why Kippo was easily detected as a honeypot by attackers was due to it only had two possible actions when a command was typed: accepted or denied. The main disadvantage of this proposal is that the proposed system is not fully operational, so there are no performance measures to prove its effectiveness.

On the other hand, the research undertaken by Kheirkhah, Mehdi, Jahanshahi & Acharya (2013) is based on the study of brute-force SSH attacks received by six different servers from a university network using the Kojoney honeypot. This is a Low-Interaction Honeypot (LIH) that simulates an SSH server. After carefully studying data retrieved from honeypots, the authors reflected in their research some similarities between recorded traffic. The researchers were able to differentiate three categories based on the entered commands. The first category consists of all successful session attempts, without the execution of subsequent commands. The purpose of this category could be the generation of new dictionaries that will be used in later brute-force attacks. The second category is composed by sessions in which attackers make use of commands only to obtain system information, such as *uname -a, proc, cpuinfo, uptime*, etc. Finally, the third category is formed by the rest of the sessions where introduced commands have a higher threat level to the system; for example, the download and execution of malware. In addition, amongst all commands registered in various honeypots, the authors stated that the 20 most repeated commands from attackers intend to provide information about the system.

An interesting study is the one presented by Fraunholz, Krohmer, Anton & Schotten (2017), where an SSH-based MIH (Kippo in this case) was configured, offering Telnet and SSH services. With this honeypot, data of attack sessions were captured to use them for statistical and behavioural analysis, such as the attacks distribution and different IPs of attackers, origin countries, anonymisation used services or skill levels of an adversary. In addition, this paper presents various automatic learning techniques that are capable of identifying unique types of sessions. Specifically, the authors proposed three different approaches to classify each session of attacks. First, the authors attempted to classify sessions according

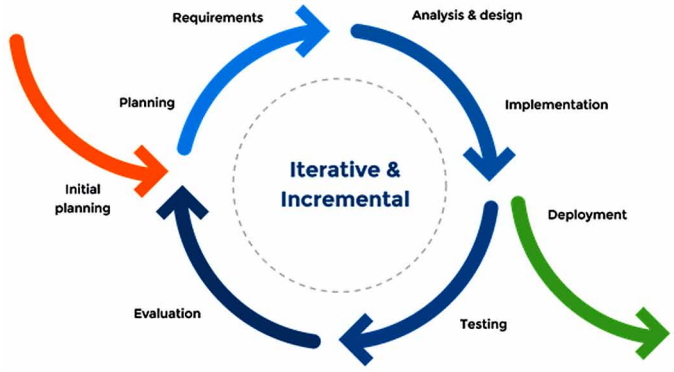
Figure 4. Metrics to classify attacker's skills

Feature	Example
Delete Commands	history -c
Check Presence	w, who, last, whoami
Delete Malware	rm (-rf) /tmp, rm (-rf) ./
Check System	ifconfig, netstat, cat /proc/ /etc/, lsmod, ps
Edit Config. File	nano/pico/vi/vim/> /etc/
Change System	nano/pico/vi/vim/> /usr/
Change Password	passwd
Manipulate Users	adduser, useradd, deluser, userdel
Manipulate Firewall	iptables, nfttables
Check Env. Variables	echo $

to the length and number of commands. However, due to the small variation in these values, no valid classification could be derived. The second approach was to classify attacks based on the attacker's skill levels, i.e., from a predefined metric the attacker received one point for each of the commands executed that belonged to the list (see Figure 4). Therefore, the higher the total number of points attackers gets, the more expert attackers were. As with the previous approach, this proposal did not obtain optimal results because most of the traffic captured by the honeypot belonged to bots, being the skill level quite uniform. Finally, the third approach was based on generating clusters from two different sets of features. On the one hand, the authors proposed to cluster from the credentials used at the beginning of each session. On the other hand, clustering was proposed based on the commands introduced by attackers. After conducting the appropriate tests, the authors stated that clustering-based approaches worked quite well in terms of generating different groups, while the skills-based approach failed due to honeypot configuration. Note that the authors did not provide graphs where to observe the represented clusters or any measure of performance, beyond indicating the similarity of each cluster with the rest.

On the other hand, in Melese & Avadhani (2016), the authors implemented in their research a Kippo honeypot to analyse SSH attacks. The main objective of this study is the statistics compiled from generated logs during sessions, as well as the conclusions that the authors contributed after studying these results. After carefully analysing the results, the authors described a sequence of common steps that most attacks carry out. First, attackers guess the credentials using automated dictionary-based attacks. Note that all login attempts with username and password were stored by Kippo, allowing through this research that future users avoid using such pairs. Amongst the most repeated credentials were default passwords such as admin, root, 123456, etc. After logging into the simulated SSH server, almost all attackers began their intrusion downloading tools from external servers. Next, attackers began their software installation and configuration tasks, and the most repeated shell commands were recorded. In addition, the authors tracked the location of attackers based on their IP addresses, as well as the SSH client used during each session. In the case of IP addresses, most attacks were originated from China and U.S., while the main used SSH client was SSH-2.0-libssh2. Finally, the authors used the Dionea honeypot (Sochor & Zuzcak, 2014) to capture and analyse possible malware that attackers tried to download. The final results achieved by the authors could be improved in the case of using High-Interaction Honeypots (HIH), which allow

Figure 5. Iterative and incremental cycle



gathering more sophisticated information that is not possible to collect with MIHs. The main disadvantage is that there is currently no SSH-based HIHs that continue to be updated on a regular basis.

3. METHODOLOGY

The work addressed in this chapter has been carried out following an iterative and incremental methodology divided into several blocks (see Figure 5). First, the analysis process on the current state-of-the-art about SSH honeypots. Second, the investigation of the most used techniques to classify attacks or threats. Third, the design and implement of a system that covers the main disadvantages of the current state-of-the-art, and finally, the system evaluation through several public datasets and a pool of specific ML classification techniques. Each block has been reiterated throughout the investigation duration, as a sample of this can be seen three experiments performed (see Section 5), addressing in each of them a new iteration in the iterative and incremental cycle.

In the first stage, an initial planning was carried out, where early steps of the research were established. Amongst them, it was the selection of an SSH-based MIH, specifically Cowrie. Also, to identify the threats registered by the honeypot and to visualise graphs where the most peculiar features about registered attacks are shown. After knowing the initial planning, the next step was the in-depth analysis of the state-of-the-art related to the classification of attacks and SSH honeypots. At the same time, an investigation of public datasets (Zkik, Orhanou & El Hajji, 2017) Cowrie available on the Internet was initiated. Next, the design and implementation of the system was undertaken. To this end, it was necessary to determine the features of the dataset and the most appropriate ML classification algorithms to obtain and analyse the results. Once the ML algorithms were determined, the next step was to verify whether the developed system was capable of identifying and classifying attacks from the selected feature set. After carrying out the first experiment, the obtained results did not reach the expected objectives, therefore we carried out a new iteration cycle with the main purpose of improving the system precision.

Prior to the analysis and redesign phase of the first proposed system, it was necessary to carefully plan the steps to be executed in this second iteration, in order to reduce the shortcomings presented in previous phase. Subsequently, it was derived for second time to the analysis and design phase, as well as to the implementation and evaluation. The obtained results in this second iteration improved satisfactorily

the ones gathered during the first iteration, however, after reading new state-of-the-art researches we were aware that these results could be improved with a new iteration. For this reason, it was decided to start a new iteration of the cycle with the new objectives to fulfil. Among them, the insertion of a new set of features or the selection of the Scriptzteam dataset. Note that the rest phases of the iterative and incremental cycle were carried out without any novel changes to highlight.

The main reason for using this type methodology lies in the fact that, unlike other fields of research, projects in this type of engineering undergo constant changes throughout their stages, from new requirements to necessary modifications during the development or evaluation phase. A clear example of this are the three experiments carried out to refine the proposed system.

The accomplishment of the previous iterations was defined to achieve the primary objective of this work, the design and implementation of an intelligent system capable of detecting and classifying the attacks received in an SSH-based MIH, specifically Cowrie. This main objective is composed of the following sub-objectives:

§ Study of the current research on SSH honeypots, as well as the search and analysis of public datasets with logs registered by Cowrie.
§ Visualisation and subsequent analysis in Kibana (Sharma, 2017) of the obtained features from the results generated by attackers, through their interaction in the registered sessions.
§ Determination of relevant features to generate a dataset capable of identifying and classifying each stored session.
§ Study of the most suitable ML techniques to maximise the expected results.
§ Use of ML techniques and dimensionality reduction that allow to link and classify similar behaviours amongst all registered sessions.
§ Careful analysis and discussion of the obtained results in each of the experiments undertaken.

4. RESULTS

This section explains the design and implementation decisions of the threat classification system proposed. Steps taken to acquire features that will make up the dataset, the decisions made regarding to ML algorithm and, finally, different experiments that have been carried out with the different sets of features and datasets.

a. Data Visualisation in Kibana

Before showing and analysing statistics and data about the registered sessions, it is necessary to store each session in ElasticSearch. Through this search engine, Kibana will allow us to make a large number of requests to the stored data, thus allowing to visualise sessions characteristics. To carry out the deployment of ElasticSearch and Kibana applications, it was determined to use the Docker tool that automates the applications deployment, packaged in software units, called *containers*, that will provide greater speed, simplicity and low consumption of resources.

Once all data are recorded in ElasticSearch, Kibana is used to display certain relevant data graphically. Note that the statistics shown below are related to the Scriptzteam dataset (Scriptzteam, 2019) by way

Figure 6. Combination of most used credentials in Scriptzteam dataset

username.keyword: Descending	password.keyword: Descending	Count
admin	admin	6,263
root	support	5,333
1111	1111	5,065
admin	1111	4,900
root	root	4,666
user	1234	4,438
root	admin	3,445
root	system	2,979
root	123456	2,639
admin	1234	2,375

of example, given that it will be the last dataset on which the validation tests of the system presented will be carried out.

Next, we show some graphics that provide the most outstanding information in the Scriptzteam dataset logs. First, the user/password pairs most commonly used to access the SSH honeypot are shown in Figure 6. As shown, default passwords such as *root/admin*, *admin/password*, *admin/admin* or *root/123456* are in the top 10 most used credentials, because nowadays many users do not yet change the default passwords that devices bring from factory, so the probability of accessing a system with such credentials remains high. For this reason, attackers try to access routers or firewalls with established default credentials. In addition, on the basis of this information, an in-depth search could be performed on user/password dictionaries most commonly used by attackers. For example, in the case of brute-force attacks generated by botnets, possible relationships could be established between the credentials, their relationship with the used dictionaries and possible links with IP addresses that use those credentials.

Figure 7. Most used SSH clients in the Scriptzteam dataset

version.keyword: Descending	Count
SSH-2.0-PUTTY	219,670
SSH-2.0-libssh2_1.6.0	43,180
SSH-2.0-libssh2_1.4.3	18,276
SSH-2.0-libssh-0.2	18,051
SSH-2.0-OpenSSH_7.3	16,848
SSH-2.0-Granados-1.0	12,380
SSH-2.0-libssh-0.6.0	6,740
SSH-2.0-libssh-0.1	6,696
SSH-2.0-sshlib-0.1	6,649
SSH-2.0-libssh2_1.7.0	5,334

Figure 8. Heat map with different IPs origin of proxies in Scriptzteam dataset



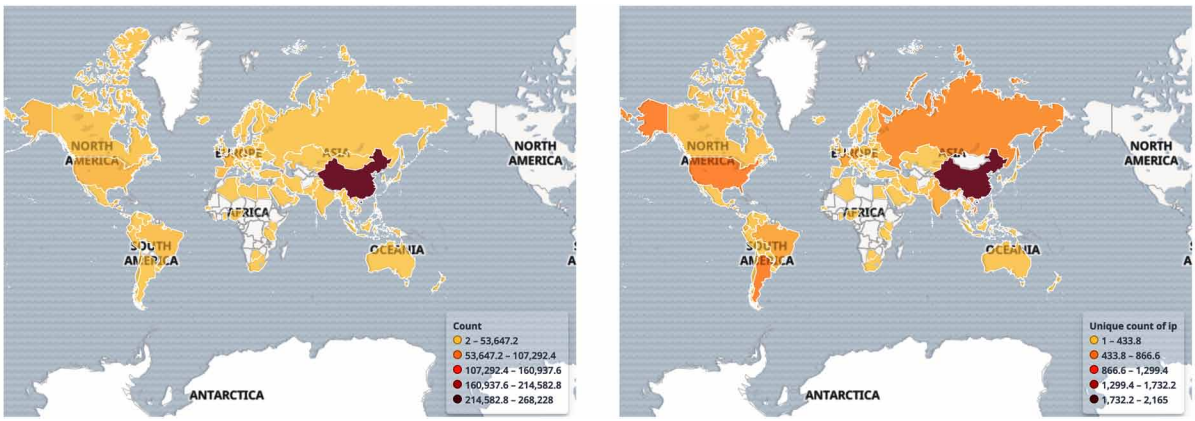
On the other hand, in Figure 7, it can be seen the SSH clients most used by attackers when they try to access the honeypot deployed in the Scriptzteam environment. The results obtained on the most used SSH clients have certain relations with the results published by ElevenPaths in a 2018 white paper (ElevenPaths, 2019), about the most relevant characteristics of different honeypots. Therefore, the SSH client might be considered as a significant feature when linking possible sessions so that repeated threats can be identified.

As a third initial test, Figure 8 depicts a heat map generated by Kibana in which the coordinates associated with each IP of a given session are represented and displayed. In a first glance, we can see that Ireland and United Kingdom are the two states from which the greatest number of attacks come. These results are certainly unusual as the countries with the highest number of generated attacks are usually U.S., China and Russia. This fact is often stated in several state-of-the-art proposals such as the one presented by Melese & Avadhani (2016) or the report published by ElevenPaths (ElevenPaths, 2019), where the authors show that countries such as the U.S. or China are amongst the countries that generate the most attacks. Therefore, we can assume the existence of proxies operating in Ireland and the United Kingdom, which act as intermediaries between the attacker origin and the destination, so the IP addresses logged by the honeypot does not correspond to real IP addresses.

Since the above map of Figure 8 does not show consistent results with the state-of-the-art and general statistics, we decided to filter out the most repeated IP addresses coming from Ireland and United Kingdom. Figure 9a shows how intermediate proxies actually exist in both countries, and which are origin countries with most of sessions recorded. In this new heat map, it is can be seen that China is positioned as one of the countries from which the greatest number of attacks are carried out, in addition to other countries such as the south of North America which begins to show itself as more malicious in terms of actions taken by attackers.

Finally, with the main objective of providing a heat map as realistic as possible, we decided to apply a new filter that only counts all attacks carried out under the same IP address as a single attack, avoiding increasing the attacks number per country if we find ourselves in a situation of the same IP address

Figure 9. Heat map with IPs over Scriptzteam dataset; (a) Real source IPs per session, (b) Real and unique source IPs



launching repeated attacks. Figure 9b illustrates this new heat map, in which it can be seen with more intense colours (i.e., higher number of attacks) other countries such as Russia, Alaska and much of South America as countries of origin of the attacks.

b. **First Experiment**

Through the different experiments, we are intended to achieve a system capable of evaluating sessions generated in the Cowrie SSH honeypot and identify and classify, as far as possible, the attacks types recorded in sessions. The aim of this first experiment is to obtain a set of features to determine which ML technique provides the best (optimal) results, i.e., algorithms based on supervised or unsupervised techniques. It is worth noting that the public dataset used for this first experiment was Rapid 7 (Rapid 7, 2019) that establishes a default setting in their honeypots.

After carefully observing each of the logs generated when an attacker establishes a connection, we can conclude that some displayed messages were shared by all sessions, because they are part of the configuration parameters that must be established during Cowrie's configuration. Therefore, despite containing a large amount of information in session-generated logs, not all logs can be used as discriminating features between sessions. In the case of encryption algorithms, for example, Cowrie determines the enabled algorithms in its configuration, and it is not possible to use an algorithm not previously configured. Thus, attackers will not be able to choose an encryption algorithm considered as in case it is not amongst the set of predefined algorithms. Consequently, after analysing possible actions that any attacker could perform without depending on Cowrie's configuration, we concluded that the command set introduced in each session would be the most relevant and discriminating features when classifying threats generated in each session.

As a consequence of the wide variety of commands that can be introduced by attackers, we decided to shorten the generated commands so as to reduce the set of different dimensions. Some examples of how the commands was shortened are the following. In the case of commands with a multitude of arguments, e.g., *echo -e -n 'bin/cowrie stop'*, they were reduced to *echo -e*. In the case of commands with paths to directories or files, e.g., *cd /var/tmp/* or *wget* http://86.101.34.88/Sbin.sh, they were reduced

to *cd /var/* and *wget* http://86.101.34.88/, respectively. It is true that this decision entails a loss of the information provided by the attacker, however, the amount of information that is discarded would only be essential in the case of wanting to detect very specific attacks of a particular type of malware, and not by identifying attacks generically that it is our target.

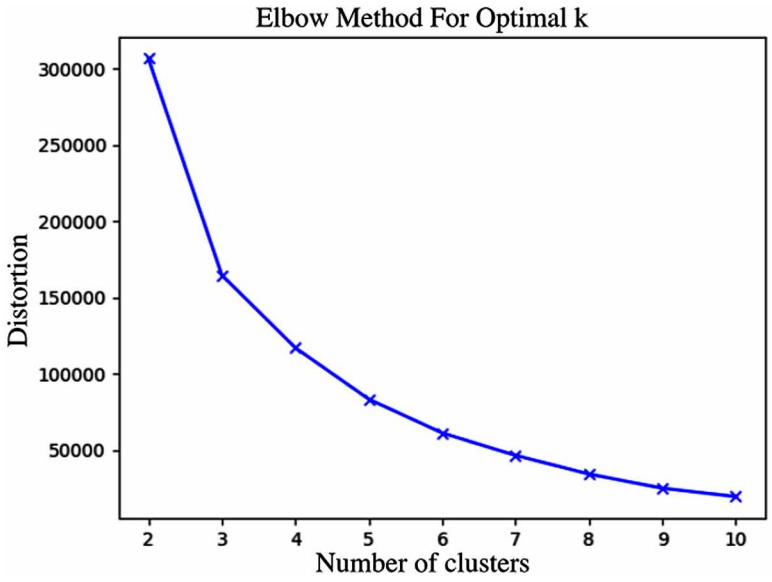
Given that the dataset was not labelled beforehand, as a first test, and in order to analyse whether there was a high relationship between all recorded sessions, it was thought to apply a clustering algorithm, in particular k-means (Hartigan & Wong, 1979), in order to visualise whether data could be partitioned into *k* groups from features that commands originated in each session could provide. K-media is a grouping method, which aims at partitioning a set of *n* observations into k-groups in which each observation belongs to the group whose mean value is closer.

A first obstacle we had with this approach was the visualising of the data provided by the k-means algorithm, given that it had a high number of dimensions (each of the possible recorded commands) and it was not possible to graphically represent the points assigned to each session. After investigating how to visualise this data, it was found out the possibility of applying dimensionality reduction techniques, which aim to maintain the distances between represented points in space and densities between different groups. In ML-based solutions, the dimensionality reduction techniques attempt to project a set of high-dimension vectors into a much lower dimensionality space, while preserving relevant information from the overall data structure. In addition to the obvious advantages of data compression, one of its main objectives is to provide an easy way to visualise the intrinsic structure of projected samples. Likewise, through reduction of the dimensions number it is possible to contribute a minor overfitting to the ML algorithm.

In a first instance, the dimensionality reduction algorithm called t-Distributed Stochastic Neighbor Embedding (t-SNE) was used (van der Maaten, 2014). This algorithm is a non-linear technique for dimensional reduction that is particularly suitable for visualisation of high-dimensional datasets. Although, t-SNE can be used as a technique to visualise data when there are a large number of dimensions, however, it should not be used as a dimensional reduction technique, since different characteristics of clusters can vary between different executions, including distance between points represented or clusters density. For this reason, an investigation into new dimensionality reduction techniques was carried out, which would allow both data visualisation in circumstances of multiple dimensions and enable the dimensions reduction. Finally, a novel multiple learning technique for dimensional reduction called Uniform Manifold Approximation and Projection (UMAP) was selected (McInnes, Healy & Melville, 2018). The UMAP algorithm is as competitive as t-SNE in visualisation quality, which can retain larger data structures providing superior runtime performance. In addition, UMAP has no computational constraints on embedded dimensions, so it is feasible as a general-purpose dimensional reduction technique for automatic learning.

After determining a dimensionality reduction technique according to data needs, the next step was to execute different instances of the clustering algorithm to confirm whether grouping was possible. To this end, it was necessary to apply One Hot Encoding (Rodríguez, Bautista, González & Escalera, 2018) to the selected features, which corresponds to a pre-processing step that is applied to categorical data in order to convert them into a non-orderly numerical representation, so that they can be properly interpreted by the ML algorithms. Then it was necessary to calculate the optimal number of clusters that are grouped (clustered) within the dataset. The elbow method was used for this purpose. This method was applied to a set of raw logs of more than 75 GBs, generating a total set of 80 features. Next, we can observe the output of this method in Figure 10.

Figure 10. Elbow method

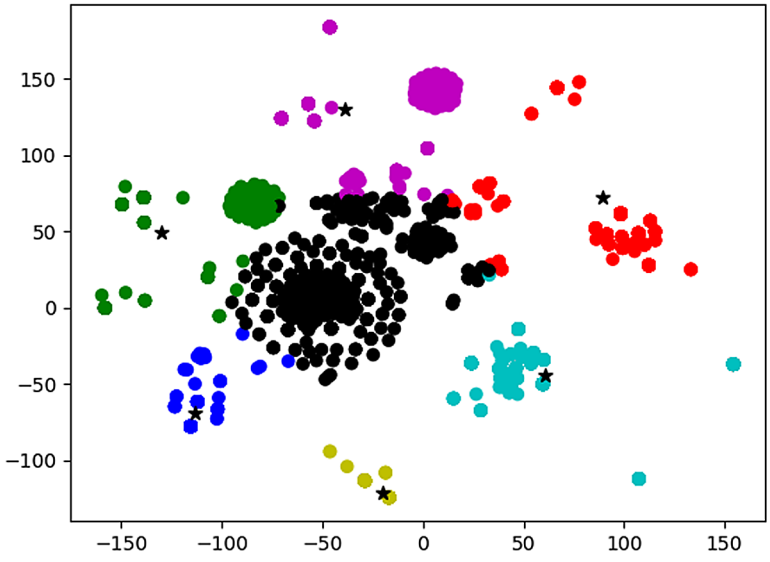


As can be seen in Figure 10, the optimal *k* for the dataset is determined between 6 and 7 clusters. To verify the optimal number of clusters, along with the rest of parameters that make up the k-means algorithm, a tuned hyperparameter technique was applied. This technique consists in appropriate parameters selection of the ML algorithm for the dataset provided. Once the optimal parameters configuration was selected, both for dimensionality reduction technique (UMAP) and clustering algorithm (k-means), the following groupings were obtained as shown in Figure 11.

Figure 11 shows how data that make up the dataset generated from the Cowrie instances, deployed in the Rapid 7 environment, cannot be precisely partitioned into *k* groups, since there is overlapping of vectors between different clusters. Given that a correct pre-processing of data was done, this being one of main reasons why clustering algorithms can produce erroneous solutions, the impossibility of grouping the dataset into properly separated clusters may be due to other factors. One of them is the inadequate selection of relevant features to determine the clusters and differentiate between attacks. Another possible cause is the nature of log set, since the Cowrie honeypot used by Rapid 7 was incorrectly configured, which might affect the generation of a limited dataset. As a consequence of the inability to apply clustering, and to determine whether the reason was due to an inappropriate selection of the ML technique, we decided to make use of classification algorithms, keeping as features the same ones that were selected for k-means algorithm.

Since data that make up the dataset are not pre-tagged, it was necessary to generate a vector label according to a predefined criterion. Before tagging vectors, it was necessary to carefully analyse the variety of commands that were executed by attackers. Once all the introduced commands had been observed, as well as the reiteration of each one of them, we decided to define a labelled set based on threat levels represented by the execution of commands in each session. To do this, three threat levels were defined based on severity to which the system could be subjected in case certain commands were executed:

Figure 11. Representation of the sessions stored in the Rapid 7 SSH honeypot through clustering



- **Level 1**: It refers to the most critical level. This is made up of sessions that attempt to both the download and execution of suspicious software, the removal of critical files from system and the suspension or shutdown of relevant system processes.
- **Level 2**: It is composed of commands that are intended to install packages on the system, compile and grant new permissions to unknown files. This level is considered an intermediate threat.
- **Level 3**: It denotes the level with the lowest threat and is composed of all commands that aim to acquire information about the current system, in order to find possible vulnerabilities in the system.

Next, a search was conducted on the most specific classification algorithms for stored data characteristics. Among the different algorithms found, it was decided to make use of k-Nearest Neighbors (k-NN), Decision Tree (DT), Random Forest (RF) and Support Vector Machines (SVM), these being some algorithms with the best results for classification. Other classification algorithms such as Logistic Regression or Naive Bayes (NB) were not selected since both classifiers assign each vector to evaluate a probability of belonging to a class instead of a deterministic assignment. Moreover, in the case of the NB algorithm is not able to provide adequate results when the dataset is composed of a large number of features as occurs in this experiment.

Once the classification algorithms had been determined, the dataset, where all SSH honeypot sessions were located, was divided into a training dataset (80%) and an evaluation dataset (20%). Besides, a stratified sampling technique was used to divide the dataset in proportion to the number of vectors contained in each class. After adjusting the algorithms, each of them was trained with its training dataset and proceeded to analyse the evaluation dataset, obtaining the results shown in Table 1. Note that *Precision* is the ratio of correctly predicted positive observations to the total predicted positive observations; *Recall* is the ratio of correctly predicted positive observations to the all observations in the actual class;

Table 1. Performance measurements of the Rapid 7 dataset (experiment 1)

	Precision	Recall	F1-Score	Accuracy
k-NN	**0.4991**	**0.4662**	**0.4709**	**0.4698**
Decision Tree	0.3287	0.3298	0.3276	0.3312
Random Forest	0.3285	0.3299	0.3275	0.3309
SVM	0.3286	0.3298	0.3276	0.3311

F1-Score is the weighted average of Precision and Recall; and *Accuracy* is the most intuitive performance measure and it is simply a ratio of correctly predicted observation to the total observations.

After obtaining low performance values in all the classification algorithms, having previously adjusted each of them, the next step was to confirm that the selected features had a relevant weight and were determinant to characterise the different sessions. The Random Forest (RF) algorithm was used to show the weight assigned to each feature. As shown in Table 2, the results returned (denoted as feature's weight ϵ [0,1]) by the RF algorithm determines that ~65% of features were not relevant to determine to which class a vector belongs, as well as which were relevant selected features from the commands introduced by attackers.

Therefore, the next experiment was to perform a new investigation of possible characteristics of each session and determine which feature set may be significant to classify the threats.

c. Second Experiment

In this second experiment, several features of the previous dataset were removed and others that are considered to be of interest were added. The aim of this experiment is to select more relevant features to identify and classify cyber threats through SSH honeypot systems. In addition, a new public dataset was selected, specifically the one provided by Scriptzteam (Scriptzteam, 2019), in order to contrast the results between a correctly configured set of logs and the ones obtained when using the default configuration (Rapid 7).

Table 2. Top 10 most relevant commands for the Rapid 7 dataset

Command	Feature's Weight
cat /proc/cpuinfo	0.1019
chmod 777	0.0793
wget	0.0771
grep name	0.0568
uname –a	0.0551
cd /tmp	0.0508
cat //.nippon	0.0476
sh tftp1.sh	0.0457
*rm -rf **	0.0402
w	0.0388

Table 3. Performance measurements of the Rapid 7 dataset (experiment 2)

	Precision	Recall	F1-Score	Accuracy
k-NN	0.7674	0.8103	0.7526	0.7515
Decision Tree	0.8297	**0.8392**	**0.8205**	**0.8609**
Random Forest	0.8297	**0.8392**	**0.8205**	**0.8609**
SVM	**0.8298**	0.8391	0.8204	0.8607

First, a new research was conducted on which new features could be selected to find an association between the introduced commands and the threat levels defined above. To do this, a rule generation algorithm called A-priori (Gupta, Kochhar, Jain & Nagrath, 2019) was used. The purpose of this algorithm is to find a set of rules that comply with the characteristics and weights indicated as parameters. The use of A-priori allows us to observe which sequence of commands is commonly used, thus being able to observe the purpose of attackers. Based on the rules obtained on the total dataset, we decided to create a new set of features in relation to the consequence that has execution of each command introduced in that session, registering a value of 1 in the case of complying with this feature and 0 in the case of not complying. Specifically, the following features were defined:

- Disk reading.
- Writing to disk.
- Obtaining system information.
- Internet connections and downloads.
- Compilation or installation of programs.
- Execution of programs.
- Suspension or elimination of processes in the system.

It should be noted that the process of vector tagging remained the same as in the previous experiment. As in the previous experiment, in this new one was also necessary to apply the tuned hyperparameter technique to adjust each classification algorithm again, since the structure of the vectors was modified between experiments. After adjusting the classification algorithms, each of them was instantiated with optimal configuration parameters, the performance measurements shown in Table 3 were obtained, which belong to the same dataset used in the previous experiment, the Rapid 7 set of logs. Note that the dataset formed from the data obtained from the Rapid 7 repository is composed of the 7 features listed above.

As can be seen in Table 3, precision, recall, F1-score and accuracy have improved significantly between experiment 1 and the current one. This is mainly due to the elimination of irrelevant features from the dataset. As suspected during the beginning of the previous experiment, the very low performance measurements of the Rapid 7 dataset could be linked to a set of inappropriate features to determine the threats produced during each session, not allowing the possible link between several attacks with similar characteristics.

Once confirmed that the new proposed set of features was able to determine precisely the generated threat type in each session, from the impact triggered by the execution of the introduced commands, the next objective of this experiment is to determine how an appropriate configuration of the system affects to the Cowrie SSH honeypot. To this end, it was decided to select public logs provided by Scriptzteam

Table 4. Performance measurements of the Scriptzteam dataset (experiment 2)

	Precision	Recall	F1-Score	Accuracy
k-NN	0.9859	0.9573	0.9708	0.9765
Decision Tree	**0.9874**	**0.9574**	**0.9714**	**0.9767**
Random Forest	0.9859	0.9573	0.9708	0.9765
SVM	**0.9874**	0.9573	**0.9714**	0.9765

(Scriptzteam, 2019), since it was configured with the purpose of obstructing an early detection of decoy system by attackers, as well as to originate a set of logs whose recorded sessions are as heterogeneous as possible. In addition, this public dataset was collected more recently (until the end of 2017) than the previous set of logs, which entails the registration of newer threats.

As in the previous experiment, it was necessary to apply a tuned hyperparameter technique to adjust each classification algorithm again, since the set of vectors was modified between experiments. Once the classification algorithms were adjusted, each of them was instantiated with the optimal configuration parameters, obtaining the performance measures shown in Table 4.

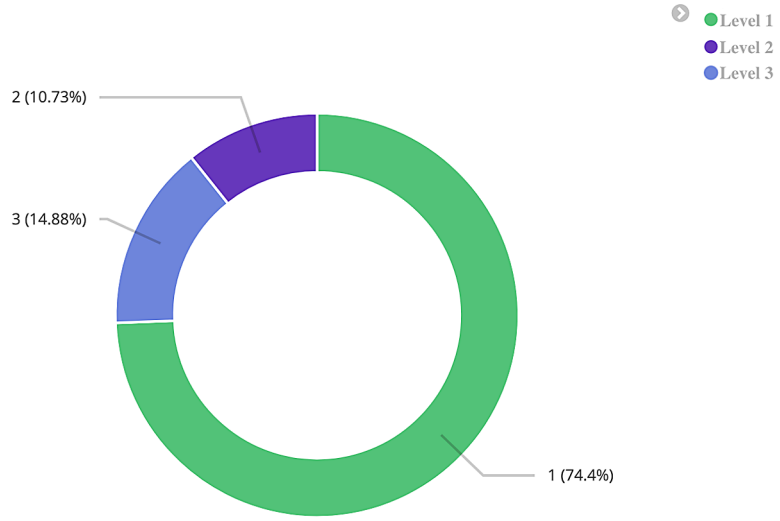
Contrasting the results provided by the performance tests carried out by the two different datasets, one with a default honeypot Cowrie configuration (Rapid 7) and another with a specific configuration (Scriptzteam), it was possible to demonstrate how the correct configuration of the decoy system is a determining factor when classifying attacks stored in the sessions. In the case of both datasets, the generated balance at each level from the labelling rules used, which were not modified since the first experiment, demonstrates that the recorded sessions play an important role in threats classification. Thus, we can conclude that not only features are decisive when it comes to obtaining classification algorithms able to detect attacks accurately, but also that an inadequate configuration of a honeypot can reduce the precision of the system considerably.

d. Third Experiment

After results obtained in the previous experiment, we decided to carry out a new research to obtain new ideas that would allow adding some differentiating features between sessions, given that the set of logs provided by Scriptzteam had a heterogeneous nature and so increasing the success level of the system presented. Although this dataset is not equally balanced between the three predefined threat levels (see Figure 12), which is an unusual situation in most scenarios, a stratified sampling technique was used to divide the dataset between training and testing. In addition, within the stratified sampling techniques, the proportional allocation has been selected, that is, the size of each stratum in the sample is proportional to its size in the population. Therefore, after the division of the general dataset between training and testing datasets, the threat levels with fewer recorded vectors are not greatly impaired. The total balance of the Scriptzteam dataset by the predefined threat levels is 5017 vectors at Level 1, 723 at Level 2 and 1003 at Level 3.

Once analysing different scientific articles related to SSH honeypots, we decided to add the following features, being some of them related to the researchers analysed in Section 2. It should be noted that these new set of features, together with the ones presented in Experiment 2, was the final set used to identify and classify cyber threats by using the Cowrie SSH honeypot.

Figure 12. Balanced dataset of Scriptzteam



§ Total time of the session.
§ Country to which the session belongs.
§ Version of the SSH client used.

As in the rest of the tests approached, it was necessary again to apply the tuned hyperparameter technique to adjust each classification algorithm, being in this last experiment only affected the set of selected characteristics, since the set of logs continued being the one provided by Scriptzteam. The performance measures obtained for the third experiment are shown in Table 5.

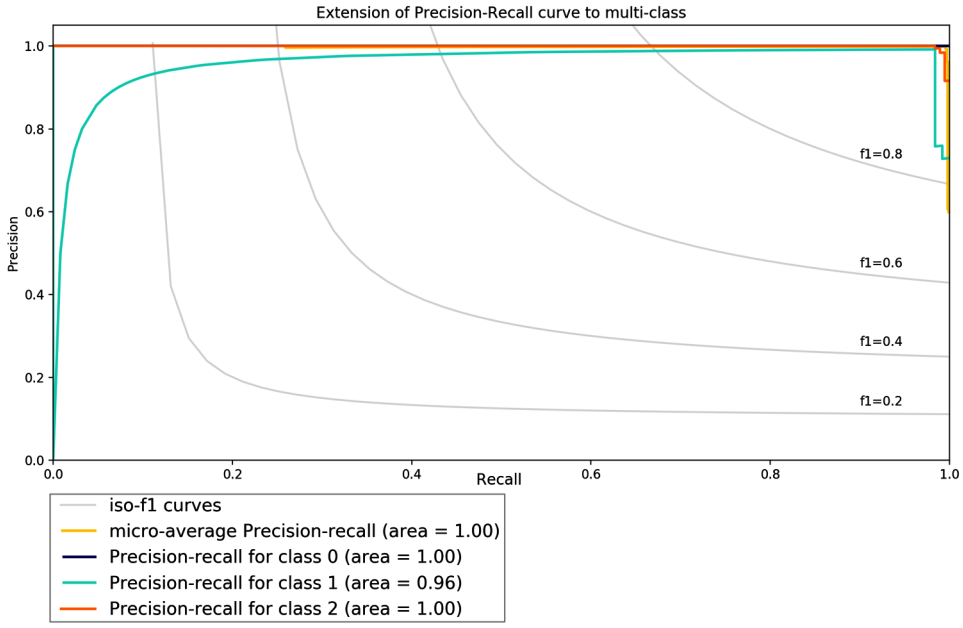
Through the obtained results, it has been possible to reduce the number of incorrectly classified attacks through the introduction of a new set of features in the dataset, thus improving performance measures provided by the classification algorithms. Specifically, the Random Forest algorithm was selected as the best algorithm to use in the proposed system given its precision and recall capability of 0.99.

Finally, a graph is depicted in Figure 13 where the precision-recall curves of the final dataset for each class are shown. This graph allows us to see from what recall we have a degradation of precision, and vice versa. The ideal solution would be a curve that gets as close as possible to the upper-right corner (high precision and high recall), being this objective achieved by means of features that have been proposed. The reason for showing a curve based on precision-recall, as opposed to the usual ROC curve (Receiver Operating Characteristic), is due to the unbalance presented by the different levels of the

Table 5. Performance measurements of the Scriptzteam dataset (experiment 3)

	Precision	Recall	F1-Score	Accuracy
k-NN	0.9666	0.9642	0.9652	0.9775
Decision Tree	0.9968	0.9968	0.9968	0.9979
Random Forest	**0.9982**	**0.9973**	**0.9977**	**0.9989**
SVM	0.9917	0.9883	0.9900	0.9948

Figure 13. Precision-recall curve for final dataset on Scriptzteam data



established threats. In addition, as the dataset is characterised for being multi-class, it will be necessary to binarize each of them to be able to be visualised in the following graph (see Figure 13). Therefore, the precision-recall assigned to class 0 belongs to threat level 1, the precision-recall assigned to class 1 belongs to threat level 2 and, finally, the precision-recall assigned to class 2 belongs to level 3. As shown in Figure 13, the final set of the defined features, which is composed of features introduced in Experiment 2 together with those of Experiment 3, are able to determine exactly to which class each of the evaluated vectors belongs, providing an almost perfect ROC curve.

Through the three experiments approached, it has been possible to identify and classify cyber threats by using an SSH honeypot system (Cowrie). For this purpose, the actions that involve the execution of commands in the system, the origin country, and, finally the SSH client and the total time of a session have been selected as features. Finally, four classification algorithms have been used that have been providing similar performance measures, however, the Random Forest algorithm obtains the best performance when detecting and classifying attacks, generating an accuracy of 0.998 and a recall of 0.997.

5. DISCUSSION

The proposed system in this chapter is a novelty in the state-of-the-art since there is no other work that combines the same set of features, uses the most current SSH-based MIH (Cowrie) and provides the performance measures we obtained. As shown in Table 6, despite the advances achieved by the previous solutions, a series of shortcomings are presented. For example, the most state-of-the-art solutions make use of Kippo, which is the honeypot predecessor to Cowrie. Kippo has a set of limitations, such as a reduced number of commands that attackers can use, which were extended in its new version (Cowrie).

Table 6. Comparison of the most current solutions

Proposal	Considered Features	Honeypot's Type	ML techniques	Results
(Sharafaldin, Lashkari & Ghorbani, 2018)	Data flows	Own deployed environments	k-NN and Random Forest	Accuracy: 0.96 and 0.98 Recall: 0.96 and 0.97 F1-score: 0.96 and 0.97
(Sadasivam, Hota & Anand, 2018)	Features based on TCP streams	Kippo	J48 and PART	Precision: 1.0 Accuracy: 0.99 F2-Score: 0.9420
(Sadasivam, Hota & Anand, 2018b)	TCP flows	Kippo	-	-
(Dowling, Schukat & Melvin, 2017)	-	Kippo	-	-
(Fraunholz, Krohmer, Anton & Schotten, 2017)	Length and number of commands, attacker's skill level	Kippo	K-means	5 clusters were generated.
(Melese & Avadhani, 2016)	Shell commands	Kippo	-	-
(Pauna & Bica, 2014)	Attacker's command	Heliza	Reinforcement learning algorithms	-
(Kheirkhah, Mehdi, Jahanshahi & Acharya, 2013)	Attacker's command	Kippo	-	-
Our proposed system	Consequences of executing commands, total session time, SSH client, country.	Cowrie	Random Forest	Precision: 0.9982 Recall: 0.9973 Accuracy: 0.9977 F1-Score: 0.9989

Therefore, Kippo provides less accurate detection than Cowrie when identifying attackers based on the commands entered in their sessions.

In this sense, the proposed solution in this work seeks to improve existing solutions. To do this, we use Cowrie as an MIH, which is the today's most mature honeypot, and we analyse the logs generated by it to determine which features would be most relevant ones for identifying and classifying cyber threats. In addition, we utilise different supervised and unsupervised learning techniques to compare and analyse the results obtained. As a novelty with respect to other state-of-the-art research, this investigation consists of the research, design and implementation of a system capable of classifying attackers' behaviour from actions carried out during their sessions in a protected and emulated environment (Cowrie). In addition, this work makes use of a set of public datasets, which were recently published and is composed of a set of current threats. Finally, the proposed system extracts the most relevant characteristics of each session, contributing a layer of intelligence to Cowrie. It should be noted that this research does verify (through three experiments) that the characteristics obtained from the visualisation of the logs can be used to determine what type of threats are registered in the honeypot.

As shown in Table 6, the work presented by (Sharafaldin, Lashkari & Ghorbani, 2018) is able to obtain a relevant performance measurement through data flows, however, the authors have the vectors tagged because they generated each attack beforehand. However, a honeypot only generates records without tags and users are who need to determine a feature set within which honeypot provides.

On the other hand, both Sadasivam, Hota & Anand (2018b), Dowling, Schukat & Melvin (2017), Kheirkhah, Mehdi, Jahanshahi & Acharya (2013) and (Melese & Avadhani, 2016) do not provide an intelligence layer using logs recorded by honeypots. The authors only research and show statistics about which are the main attacker's behaviours, however, that information is not contrasted with real experiments. In some cases, the authors use IP addresses as a possible feature to identify attackers, but do not consider that an IP address might not be real as our proposal does, that is, an IP address used through a proxy to mask real addresses of attackers.

As system limitations, Cowrie continues to be an MIH, which has a delimited command set, though larger than its predecessor honeypots. Therefore, the classification of attacks cannot be very precise since there is not a large set of features to differentiate, for example, amongst different types of malware. In addition, the available SSH-based HIHs are outdated, which could provide a risk to our real systems if we deployed them, so opting for an MIH is the best choice for safety reasons.

6. CONCLUSION AND FUTURE WORK

This work proposes a system for the identification and classification of cyber threats through SSH-based honeypot systems. With the ideas presented, it has been possible to address the relevant information extraction from logs generated by the Cowrie honeypot. Through this information, three threat levels were generated to classify attacker's session based on a set of features. The presented system complements the different traditional threat detection techniques (IDSs, antivirus, etc.) with a novel system capable of detecting previously unknown threats, while providing a higher security level. In order to provide an intelligence layer to the proposed system, several supervised ML algorithms based on classification have been used to evaluate each session registered by the Cowrie honeypot. The visualisation of characteristics belonging to sessions through the Kibana data viewer, as well as different experiments conducted through several ML algorithms, dimensionality reduction techniques and the use of a rule generation algorithm, have allowed to obtain and validate an optimal solution capable of identifying and classifying cyber threats.

As future work, we plan to examine new High-Interaction Honeypots (HIH), both SSH and other families oriented to services or applications in order to obtain greater user interaction. In addition, the undertaken study will be used as a basis for the subsequent generation of detection signatures, with which to update IDSs based on misuse detection or the labelling of datasets for detection engines retraining of systems based on anomaly detection. Finally, we plan to generate a new set of features from MIH and HIH honeypots, as well as using additional Machine Learning o Deep Learning algorithms to test the efficiency classification of attacks with new improvements added. It should be noted that in order to obtain a new set of features with MIHs, it is intended to generate a specific parser that allows new information to be obtained from logs generated by the MIHs and that information is not available through the basic information they provide.

REFERENCES

Barrett, D. J., Silverman, R. E., & Byrnes, R. G. (2005). *SSH, the Secure Shell: The definitive guide*. O'Reilly Media.

Depren, O., Topallar, M., Anarim, E., & Ciliz, M. K. (2005). An intelligent intrusion detection system (IDS) for anomaly and misuse detection in computer networks. *Expert Systems with Applications*, *29*(4), 713–722. doi:10.1016/j.eswa.2005.05.002

Dowling, S., Schukat, M., & Melvin, H. (2017). A ZigBee honeypot to assess IoT cyberattack behaviour. In *Proceedings 28th Irish Signals and Systems Conference*, 1-6. 10.1109/ISSC.2017.7983603

ElevenPaths. (2019). Honeypotting, un oído en Internet. Retrieved October 9, 2019, from https://empresas.blogthinkbig.com/informe-ciberseguridad-honeypotting-un-oido-en-interne/

Fraunholz, D., Krohmer, D., Anton, S. D., & Schotten, H. D. (2017). Investigation of cyber-crime conducted by abusing weak or default passwords with a medium interaction honeypot. In *2017 International Conference on Cyber Security and Protection of Digital Services (Cyber Security)*, 1-7. 10.1109/CyberSecPODS.2017.8074855

Gupta, B. B. (2018). Computer and cyber security: principles, algorithm, applications, and perspectives. Boca Raton, FL: CRC Press, Taylor & Francis.

Gupta, B. B., & Sheng, Q. Z. (2019). *Machine Learning for Computer and Cyber Security: Principle, Algorithms, and Practices*. CRC Press. doi:10.1201/9780429504044

Gupta, M., Kochhar, S., Jain, P., & Nagrath, P. (2019). Hybrid recommender system using A-priori algorithm. In *Proceedings International Conference on Sustainable Computing in Science, Technology and Management*, 26-28. Academic Press.

Hartigan, J. A., & Wong, M. A. (1979). Algorithm AS 136: A k-means clustering algorithm. *Journal of the Royal Statistical Society. Series A (General)*, *28*(1), 100–108.

Jiang, F., Fu, Y., Gupta, B. B., Lou, F., Rho, S., Meng, F., & Tian, Z. (2018). Deep learning based multichannel intelligent attack detection for data security. *IEEE transactions on Sustainable Computing*.

Jorquera, J. M., Sánchez, P. M., Fernández, L., Huertas, A., Arjona, M., De Los Santos, S., & Martínez, G. (2018). Improving the security and QoE in mobile devices through an intelligent and adaptive continuous authentication system. *Sensors (Basel)*, *18*(11), 3769. doi:10.339018113769 PMID:30400377

Keane, J. K. (2019). Kojoney2 SSH medium-interaction honeypot. Retrieved October 5, 2019, from https://github.com/madirish/kojoney2

Kheirkhah, E., Mehdi, S., Jahanshahi, H. A., & Acharya, H. (2013). An experimental study of SSH attacks by using honeypot decoys. *Indian Journal of Science and Technology*, *6*(12), 5567–5578.

McInnes, L., Healy, J., & Melville, J. (2018). UMAP: Uniform manifold approximation and projection for dimension reduction. *The Journal of Open Source Software*, *3*(29), 861. doi:10.21105/joss.00861

Melese, S. Z., & Avadhani, P. S. (2016). Honeypot system for attacks on SSH protocol. *International Journal of Computer Network and Information Security*, *8*(9), 19. doi:10.5815/ijcnis.2016.09.03

Melese, S. Z., & Avadhani, P. S. (2016). Honeypot system for attacks on SSH protocol. *International Journal of Computer Network and Information Security*, *8*(9), 19. doi:10.5815/ijcnis.2016.09.03

Olakanmi, O. O., & Dada, A. (2019). An Efficient Privacy-preserving Approach for Secure Verifiable Outsourced Computing on Untrusted Platforms. [IJCAC]. *International Journal of Cloud Applications and Computing*, *9*(2), 79–98. doi:10.4018/IJCAC.2019040105

Oosterhof, M. (2019). Cowrie SSH honeypot (based on Kippo). Retrieved October 5, 2019, from https://github.com/cowrie/cowrie

Padayachee, K. (2014). Aspectising honeytokens to contain the insider threat. *IETF Information Security*, *9*(4), 240–247. doi:10.1049/iet-ifs.2014.0063

Pauna, A., & Bica, I. (2014). RASSH-Reinforced adaptive SSH honeypot. In *Proceedings 10th International Conference on Communications*, 1-6.

Rapid 7. (2019). Public data of the honeynet deployed by the company Rapid 7 in Heisenberg. Retrieved October 7, 2019, from https://opendata.rapid7.com/heisenberg.cowrie/

Rodríguez, P., Bautista, M. A., González, J., & Escalera, S. (2018). Beyond one-hotencoding: Lower dimensional target embedding, *Image and Vision Computing*, 75, 21-31.

Rosenberg, J., Coronel, J. B., Meiring, J., Gray, S., & Brown, T. (2019). Leveraging ElasticSearch to improve data discoverability in science gateways. In Practice and Experience in Advanced Research Computing on Rise of the Machines Learning, 19. doi:10.1145/3332186.3332230

Sadasivam, G. K., Hota, C., & Anand, B. (2018). Detection of severe SSH attacks using honeypot servers and machine learning techniques. *Software Networking*, *2017*(1), 79–100. doi:10.13052/jsn2445-9739.2017.005

Sadasivam, G. K., Hota, C., & Anand, B. (2018). Honeynet data analysis and distributed SSH brute-force attacks. In Towards Extensible and Adaptable Methods in Computing, 107-118. doi:10.1007/978-981-13-2348-5_9

Scriptzteam. (2019). Public repository in GitHub. Retrieved October 8, 2019, from https://github.com/scriptzteam/REALTiME-SSH-HoneyPot_v1.1

Sharafaldin, I., Lashkari, A. H., & Ghorbani, A. A. (2018). Toward generating a new intrusion detection dataset and intrusion traffic characterization. In *Proceedings 4th International Conference on Information Systems Security and Privacy*, 108-116. 10.5220/0006639801080116

Sharma, V. (2017). Cap. getting started with Kibana. *Beginning Elastic Stack, Apress*, 29-44.

SOC eSentire. (2017). Annual threat report. Retrieved October 5, 2019, from https://www.esentire.com/resource-library/2017-annual-threat-report

Sochor, T., & Zuzcak, M. (2014). Study of internet threats and attack methods using honeypots and honeynets. In *Proceedings International Conference on Computer Networks*, 118-127. 10.1007/978-3-319-07941-7_12

Spitzner, L. (2003). *Honeypots: Tracking hackers* (p. 1). Reading: Addison-Wesley.

Valli, C., Rabadia, P., & Woodward, A. (2013). Patterns and patter - An investigation into SSH activity using Kippo honeypots. In *Australian Digital Forensics Conference*, 141-149.

Van der Maaten, L. (2014). Accelerating t-SNE using tree-based algorithms. *Journal of Machine Learning Research*, *15*(1), 3221–3245.

Zhang, M., Wang, L., Jajodia, S., Singhal, A., & Albanese, M. (2016). Network diversity: A security metric for evaluating the resilience of networks against zero-day attacks. *IEEE Transactions on Information Forensics and Security*, *11*(5), 1071–1086. doi:10.1109/TIFS.2016.2516916

Zkik, K., Orhanou, G., & El Hajji, S. (2017). Secure mobile multi cloud architecture for authentication and data storage. [IJCAC]. *International Journal of Cloud Applications and Computing*, *7*(2), 62–76. doi:10.4018/IJCAC.2017040105